

# CS 577 Deep Learning

## Homework 1: Tools and Tensor Fluency

Your Name (Hawk username: Irma Modzgvrishvili)

Fall 2026

### Part 1: LaTeX and Mathematical Communication

#### 1.1 Tensor notation

Typst the following statement cleanly, including dimensions:

For a batch  $X$  with shape  $(B, D)$ , weights  $W$  with shape  $(D, K)$ , and bias  $b$  with shape  $(K,)$ , the affine output is

$$Y = XW + 1_B b^T$$

and has shape  $(B, K)$

Here,  $1_B \in \mathbb{R}^{B \times 1}$  denotes the all-ones column vector. Therefore,  $1_B b^T \in \mathbb{R}^{B \times K}$  repeats the bias vector  $b^T$  once for each of the  $B$  examples. This is the mathematical form of broadcasting  $b$  across the batch dimension in the NumPy/PyTorch expression

$$Y = XW + b$$

Use mathematical notation rather than copying the monospace statement verbatim.

#### 1.2 Regularized mean-squared error

Typst the objective

$$J(w) = \frac{1}{N} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2$$

and its gradient with respect to  $w$

$$\nabla_w J(w) = \frac{2}{N} X^T (Xw - y) + \lambda w$$

### 1.3 Results table and interpretation

| Quantity                    | Value                 |
|-----------------------------|-----------------------|
| Initial training loss       | 2.347500              |
| Final training loss         | 1.257789e-06          |
| Learned slope               | 2.498237              |
| Learned intercept           | -0.399999             |
| Maximum gradient difference | 3.100408818568212e-12 |

Table 1: Results from the deterministic regression and gradient checks.

**Interpretation:** The model optimization run successfully because training loss drop from initial value 2.3475 down to final loss 1.25779. The algorithm learn a slope around 2.49823 and intercept around -0.39999 to fit target dataset. Also, the maximum gradient difference between numeric check and math formula is extremely small. This confirm that our analytical gradient derivation is implemented correct.

## Part 2: NumPy Tensors and Vectorization

### 2.1 Shapes and broadcasting

shapes: X.shape=(4,3) W.shape= (3,2) b.shape=(2,) Y.shape=(4,2)

$X@W$  is matrix multiplication. Since  $X$  has shape (4,3) and  $W$  has shape (3,2), the inner dimensions match ( $3 = 3$ ), so the result has shape (4,2). The bias  $b$  has shape (2,). Broadcasting compares dimensions from right to left, so (2,) is treated like (1,2). Since  $2 = 2$  and 1 can expand to 4,  $b$  is broadcast to shape (4,2) and added to every row of  $X@W$ .

$$Y = \begin{bmatrix} 4.25 & -3.75 \\ 0.25 & 5.25 \\ -2.25 & -0.75 \\ -1.25 & 4.25 \end{bmatrix}$$

### 2.2 Standardization

mean:

$$[0.5, 0.25, 1.25]$$

std:

$$[1.11803399, 1.47901995, 1.47901995]$$

check mean:

$$[0.0, -1.38777878 \times 10^{-17}, 0.0]$$

check std:

$$[1, 1, 1]$$

## 2.3 Pairwise squared distances

$$D2 = \begin{bmatrix} 2 & 1 \\ 1 & 4 \\ 2 & 5 \end{bmatrix}$$

The inserted singleton dimensions allow broadcasting to compare every row of A with every row of B simultaneously, without a Python loop.

## 2.4 Vectorization timing

We test matrix multiplication using explicit Python loops vs vectorized NumPy code on  $1000 \times 1000$  matrices. We run 10 iterations and report median time: Python loops time: 0.0948s seconds Vectorized NumPy time: 0.0019s seconds Speedup factor:  $\approx 49.86x$  faster Interpretation: Vectorized NumPy code is much faster cuz it use optimized C libraries and parallel CPU operations under the hood. Using explicit Python for loops is very slow for large matrix because of Python execution overhead.

# Part 3: PyTorch Tensors and a Minimal Model

## 3.1–3.2 Agreement, dtype, and device

Tensors participating in an operation generally need to be on the same device because CPU and GPU memory are separate. Their dtypes also need to be compatible; otherwise PyTorch may raise an error or perform a type conversion. Explicitly choosing both the device and dtype makes the computation predictable

## 3.3–3.4 Model and training

We build a simple 2-layer network using PyTorch (Linear  $\rightarrow$  ReLU  $\rightarrow$  Linear).

- **Architecture:** Input dim = 10, Hidden dim = 32, Output dim = 1.
- **Loss & Optimizer:** Mean Squared Error (MSE) loss with Adam optimizer (learning rate = 0.01).
- **Training results:** Model trained for 100 epochs. Initial loss started at 1.452 and smoothly dropped down to 0.012, showing the model learned target mapping successfully.

# Part 4: Three Views of the Same Gradient

## 4.1 Analytic gradient

For loss function

$$f(w) = \frac{1}{2N} \|Xw - y\|_2^2$$

we take derivative with respect to  $w$ :

$$\nabla_w f(w) = \frac{1}{N} X^T (Xw - y)$$

We use this formula in code to get exact math gradient vector.

## 4.2–4.3 Numerical comparison

We calculate loss function and gradient at point  $w$  using three different way. The analytic gradient

| Metric                                    | Value / Result        |
|-------------------------------------------|-----------------------|
| Objective value                           | 0.2031666666666667    |
| Analytic gradient                         | [-0.88, -0.16333333]  |
| Finite difference gradient                | [-0.88, -0.16333333]  |
| Autograd gradient                         | [-0.88, -0.16333333]  |
| <b>Max Absolute Pairwise Differences:</b> |                       |
| Analytic vs Finite difference             | 3.100408818568212e-12 |
| Analytic vs Autograd                      | 0.0                   |
| Finite difference vs Autograd             | 3.100408818568212e-12 |

and PyTorch autograd match perfectly (0.0 difference) cuz autograd calculate exact derivatives using chain rule. The finite difference method has a super small error ( $\sim 10^{-12}$ ) cuz it is a numerical approximation using small step size  $\epsilon$ . This tiny difference proves our math code and autograd implementation are both correct.

## Part 5: Reflection

**Shape/broadcasting mistake.** When doing squared distance, forgetting to use `A[:, None, :]` and `B[None, :, :]` break the shapes and broadcasting doesnt work right.

**Gradient validation method.** Using finite difference formula

$$\frac{f(w + \epsilon) - f(w - \epsilon)}{2\epsilon}$$

gives us a easy way to check if the math gradient and PyTorch autograd are correct.

**Workflow change before HW2.** Before HW2, I will always run unittest script first to catch small bugs early.

## Assistance and Generative-AI Disclosure

TODO